



郑州大学
Zhengzhou University

第四单元 哈希函数与消息认证码

郑州大学网络空间安全学院

王志华

王志华



第4单元 哈希函数与消息认证码

提纲

CONTENTS

4.1 哈希函数概述

4.2 MD算法

4.3 SHA算法

4.4 MAC

4.5 SM3

4.6 其他哈希函数

王华

4.1 哈希函数概述



郑州大学
Zhengzhou University

4.1.4 迭代哈希函数

- ◆ 由Merkle于1989年提出
- ◆ Ron Rivest于1990年提出MD4
- ◆ 常用Hash函数均使用该结构：MD5, SHA-1
- ◆ 采用迭代结构

2022/4/24

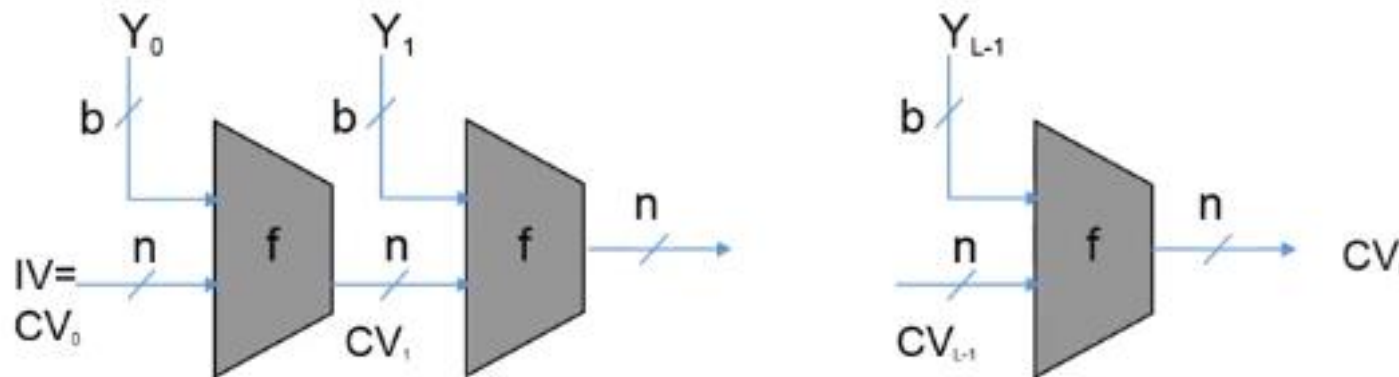


1、迭代型哈希函数的描述

- 大多数哈希函数如MD5、SHA-1，其结构都是**迭代的**：
 - 函数的**输入M**被分为**L个分组** $Y_0, Y_1, \dots, Y_{L-1}$ ，每一个**分组的长度为b比特**
 - 最后一个分组的长度不够的话，需对其做**填充**
 - 最后一个分组中还包括**整个函数输入的长度值**
 - 敌手攻击更困难，敌手若想**成功地产生假冒的消息**
 - 必需保证假冒消息的哈希值与原消息的哈希值相同
 - 必需保证假冒消息的长度也要与原消息的长度相等
- 思考：**迭代分组密码的设计思想**。

王卫华

2、迭代型哈希函数的算法流程



$CV_0 = IV = \text{initial } n\text{-bit value}$
 $CV_i = f(CV_{i-1}, Y_{i-1}) \quad (1 \leq i \leq L)$
 $H(M) = CV_L$

IV = initial value 初始值
 CV = chaining value 链接值
 Y_i = i th input block (第 i 个输入数据块)
 f = compression algorithm (压缩算法)
 n = length of hash code (散列码的长度)
 b = length of input block (输入块的长度)

思考：这种链接方式像什么？？？

密码分组链接模式**CBC**！ ！ ！

2024



2、迭代型哈希函数的算法流程

● 迭代哈希函数算法流程：

- 算法重复使用**一个压缩函数f**（有的将哈希函数也称为压缩函数，在此**用压缩函数表示哈希函数中的一个特定部分**）。通常有 **$b > n$** ，因此称函数f为**压缩函数**
- f 的输入有两项：
 - ✓ 一项是上一轮（第i-1轮）输出的**n比特值** CV_{i-1} ，称为**链接变量**
 - ✓ 另一项是算法在本轮（第i轮）的**b比特输入分组** Y_{i-1}
- f 的输出为**n比特值** CV_i ， CV_i 又作为下一轮的输入
- 算法开始时还需对**链接变量**指定一个**初值IV**
- 最后一轮输出的**链接变量**即为**最终产生的哈希值**
- 算法可如下表达：

$$CV_0 = IV$$

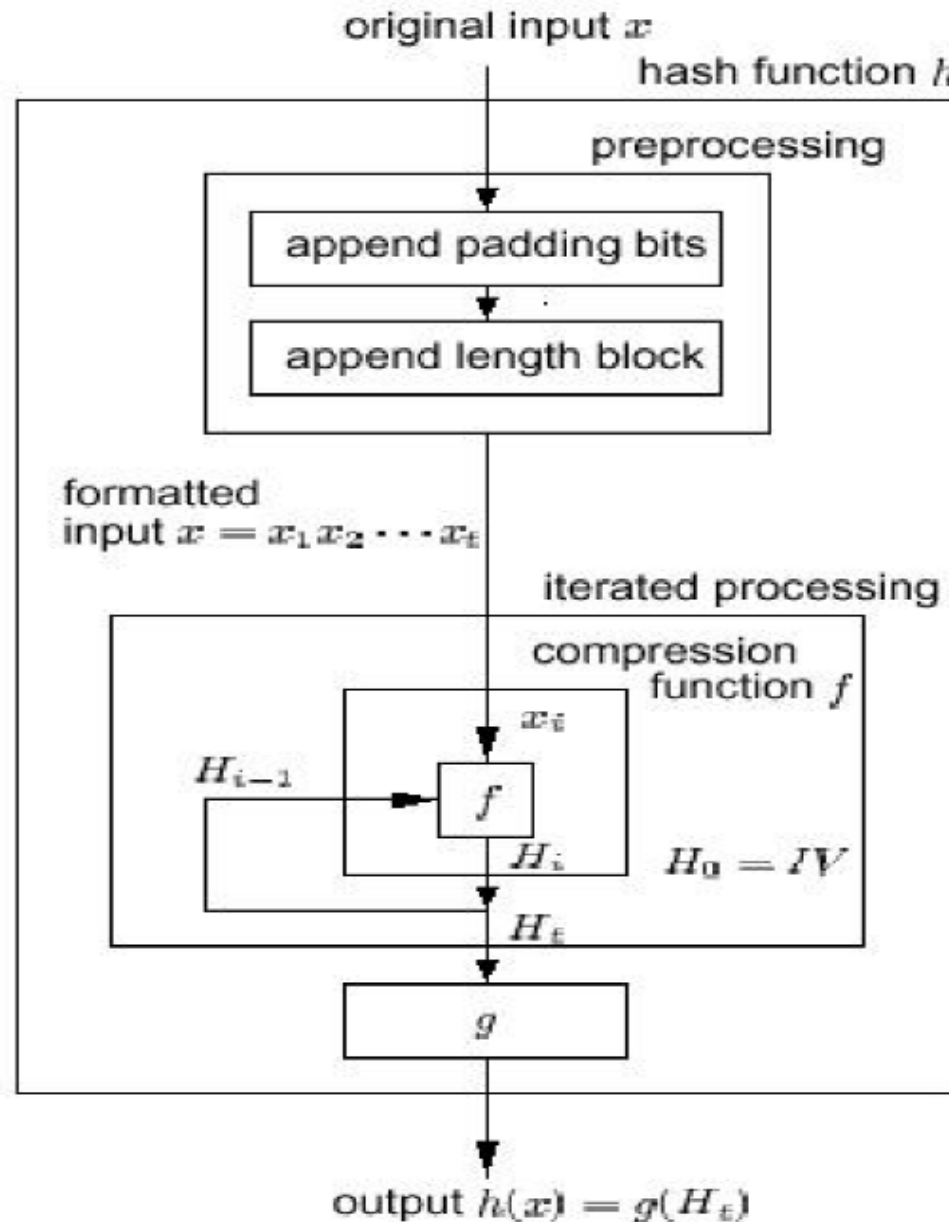
$$CV_i = f(CV_{i-1}, Y_{i-1}) \quad 1 \leq i \leq L$$

$$H(M) = CV_L$$

2022/4/24

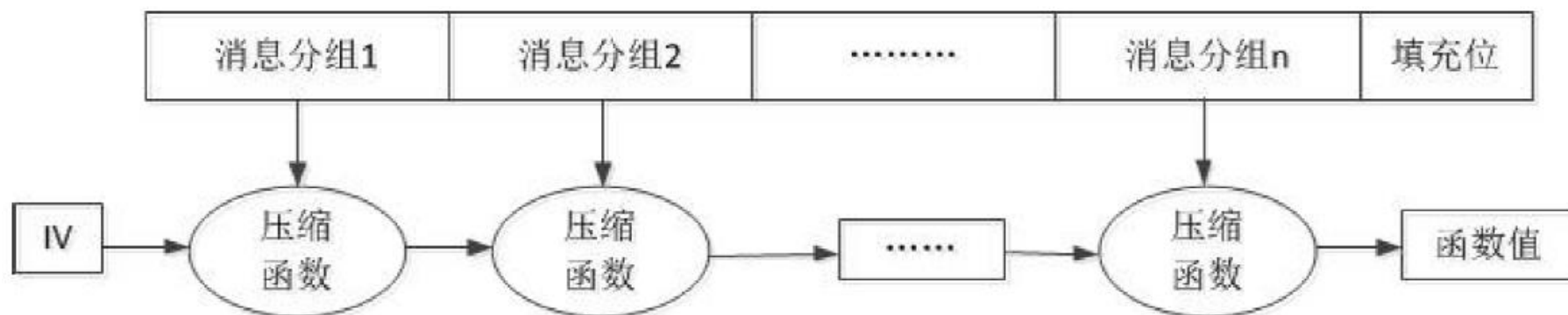


3、迭代型哈希函数的算法实现





4、迭代型哈希函数的压缩函数



王华



4、迭代型哈希函数的压缩函数

- 迭代函数中的压缩函数：
 - 算法的核心技术是**设计无碰撞的压缩函数**
 - 敌手对算法的**攻击重点**是**压缩函数的内部结构**
 - 由于压缩函数和分组密码一样是由若干轮处理过程组成，所以对压缩函数的攻击需通过对**各轮之间的位模式的分析**来进行：
 - 分析过程常常需要先找出**压缩函数的碰撞**
 - 由于压缩函数的**碰撞是不可避免的**，因此在设计压缩函数时就应保证找出其碰撞**在计算上是不可行的**

王卫华

4.2 MD算法



4.2.1 MD算法概述

●MD5 — “message digest”

- 由Rivest发明, 1992年公开
- 128比特输出
- 2004年, 已经证实MD5算法无法防止碰撞, 因此不适用于安全性认证

2024



1、MD算法的历史

- ◆ Ron Rivest于1990年提出MD4。
- ◆ 1992年，MD5 (RFC 1321) developed by Ron Rivest at MIT。
- ◆ MD5把数据分成512-bit/块，MD5的Hash值是128-bit。
- ◆ 在最近数年之前，MD5是最主要的Hash算法。
- ◆ Dobbertin在1996年找到了两个不同的512-bit块，它们在MD5计算下产生相同的散列值。
- ◆ 2004年8月17日国际密码学会议，山东大学王小云教授宣告破解了包含MD5在内的数个单向散列算法。

王立华



2、MD4和MD5

- ❖ **MD4**是麻省理工学院教授**Ronald Rivest**于**1990**年设计的一种信息摘要算法。它是一种用来测试信息完整性的密码散列函数的实行。其摘要长度为**128**位。这个算法影响了后来的算法如**MD5**、**SHA** 家族和**RIPEMD**等。
- ❖ **MD5**算法是**1991**年发布的一项数字签名加密算法，它当时解决了**MD4**算法的安全性缺陷，成为应用非常广泛的一种算法。

2024

4.2 MD算法



4.2.2 MD4算法

- 最早Ron Rivest设计了MD (Message Digest) ;
- 在1989年开发出MD2算法;
- 之后的版本为MD3, 还是不理想, 未公布;
- 为了加强算法的安全性, Rivest在1990年又开发出MD4算法。

2024



1、MD4算法概述

- MD4的特点是：
 - 对任意长度的输入，产生**128比特输出**；
 - 其安全性**不依赖于任何假设**，适合高速实现。

王华



2、MD4算法描述

❖ **MD4**算法的输入可以是任意长度的消息 x ，对输入消息按**512**位的分组为单位进行处理，输出**128**位的散列值**MD(x)**。整个算法分为五个步骤。

- 步骤1: 增加填充位
- 步骤2: 附加消息长度值
- 步骤3: 初始化**MD**缓冲区
- 步骤4: 以**512**位的分组(**16**个字)为单位处理消息
- 步骤5: 输出

2024



2、MD4算法描述

设 x 是一个消息, 用二进制表示。首先由 x 生成一个数组

$$M = M[0]M[1] \cdots M[N-1],$$

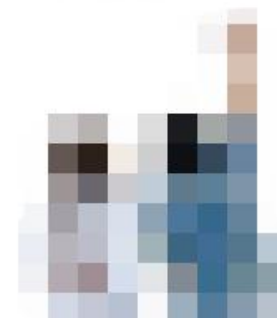
$M[i]$ 是长度为32 比特(bit)的(0,1) 序列, $0 \leq i \leq N-1$, M 由 x 生成:

① $d = (447 - |x|) \bmod 512$

② 令 l 为 $|x| \bmod 2^{64}$ 的二进制表示。 l 的长度为64 比特(bit)。
如果 l 的长度不足64 比特(bit), 则在 l 的左端添0 补足

③ $M = x || 1 || 0^d || l$

这里 $|x|$ 表示 x 的长度, $||$ 表示序列的联接,
譬如 $x||y$ 表示将序列 y 排在序列 x 的右端。



2024



2、MD4算法描述

❖ 步骤1: 增加填充位

- 在消息 x 右边增加若干比特, 使其长度与448模512同余。也就是说, 填充后的消息长度比512的某个倍数少64位。
- 即使消息本身已经满足上述长度要求, 仍然需要进行填充。
- 例如, 若消息长为448, 则仍需要填充512位使其长度为960位。填充位数在1到512之间。填充比特的第一位是1, 其它均为0。

2022/4/24



2、MD4算法描述

步骤2: 附加消息长度值

用**64**位表示原始消息**x**的长度，并将其附加在步骤1所得结果之后。若填充前消息长度大于等于 2^{64} ，则使用其**64**位。填充方法是把**64**比特的长度分成两个**32**比特的字，低**32**比特字先填充，高**32**比特字后填充。

2024



2、MD4算法描述

❖ 步骤1与步骤2一起称为消息的预处理

- 经预处理后，原消息长度变为**512**的倍数
- 设原消息**x**经预处理后变为消息

$$Y = Y_0 Y_1 \dots Y_{N-1},$$

其中 Y_i ($i=0,1,\dots,N-1$) 是**32**比特

- 在后面的步骤中，将对**512**比特的分组 Y_i 进行处理，每次**16**个 Y_i 。

2024



2、MD4算法描述

❖ 例8.1 假设消息为：

$x = \text{"abcde"} = 01100001\ 01100010\ 01100011\ 01100100\ 01100101 = (61\ 62\ 63\ 64\ 65)_{16}$, $|x| = 40 = (28)_{16}$.

- 步骤1在 x 的右边填充1个“1”和407个“0”，将 x 变成448比特的 x_1 ：

$x_1 = x \parallel 1 \parallel 0\ (407\text{个})$

$= x \parallel 800000\ 00000000\ 00000000\ 00000000\ 00000000$
 $00000000\ 00000000\ 00000000\ 00000000\ 00000000$
 $00000000\ 00000000\ 00000000.$

$= 61626364\ 65800000\ 00000000\ 00000000\ 00000000\ 00000000$
 $00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000$
 $00000000\ 00000000.$

王华



2、MD4算法描述

■ 例8.1 假设消息为:

$x = \text{"abcde"} = 01100001\ 01100010\ 01100011$
 $01100100\ 01100101 = (61\ 62\ 63\ 64\ 65)_{16},$
 $|x| = 40 = (28)_{16}.$

- 经步骤2处理后的比特串为（16进制表示）：

$x_2 = x_1 || 28(64\text{位})$
 $= 61626364\ 65800000\ 00000000\ 00000000\ 00000000$
 $00000000\ 00000000\ 00000000\ 00000000$
 $00000000\ 00000000\ 00000000\ 00000000\ 00000000$
 $28000000\ 00000000.$

王华



2、MD4算法描述

❖ 步骤3: 初始化MD缓冲区

- **MD4算法的中间结果和最终结果都保存在128位的缓冲区里，缓冲区用4个32位的寄存器表示。**
- **4个缓冲区记为A、B、C、D，其初始值为下列32位整数（16进制表示）：**
A=67 45 23 01, B=EF CD AB 89,
C=98 BA DC FE, D=10 32 54 76.
- **上述初始值以小端格式存储(字的最低有效字节存储在低地址位置)为：**
字A=01 23 45 67, 字B=89 AB CD EF,
字C=FE DC BA 98, 字D=76 54 32 10.





2、MD4算法描述

❖ 步骤4: 以512位的分组(16个字)为单位处理消息

- MD4是迭代Hash函数, 其压缩函数为:

$$H_{MD4} : \{0,1\}^{128+512} \rightarrow \{0,1\}^{128}.$$

- 步骤4是MD4算法的主循环, 它以512比特作为分组, 重复应用压缩函数 H_{MD4} , 从消息 Y 的第一个分组 Y_0 开始, 依次对每16个分组 Y_i 进行压缩, 直至最后分组 $N/16-1$, 然后输出消息 x 的Hash值。可见, MD4的循环次数等于消息 Y 中512比特分组的数目 L 。



2024



2、MD4算法描述

- ❖ 步骤5: 输出
- ❖ 依次对消息的 L 个512比特的分组进行处理，第 L 个分组处理后的输出值即是消息 x 的散列值 $MD(x)$ 。

2024



3、MD4算法实现

MD4 算法如下

- ①设 A 、 B 、 C 、 D 是四个32位的寄存器,其初值(用十六进制表示)分别为 $A=67452301$ 、 $B=efcdab89$ 、 $C=98badcfe$ 、 $D=10325476$:
- ②对 $i = 0$ 至 $N/16 - 1$ 执行第3步至第8步
- ③对 $j = 0$ 至15执行 $X[j] = M[16i + j]$
- ④将寄存器 A 、 B 、 C 、 D 中的值存储到另外四个寄存器 AA 、 BB 、 CC 、 DD 中,
 $AA = A$, $BB = B$, $CC = C$, $DD = D$
- ⑤ 执行Round1
- ⑥ 执行Round2
- ⑦ 执行Round3
- ⑧ $A = A + AA$, $B = B + BB$, $C = C + CC$, $D = D + DD$

2024



4、MD4的安全性

- MD4公布不久，一些密码学家发现：
 - 如果去掉MD4算法的第一轮和最后一轮，则算法是不安全的
 - 但当时他们并没有证明整个算法是不安全的

2022/4/24

4.2 MD算法



4.2.3 MD5算法

- 1991年，Rivest开发出技术上更为趋近成熟的MD5算法；
- 1992年8月，Rivest向互联网工程任务组（IETF）提交了一份重要文件，描述了MD5算法的原理；
- MD5由MD4、MD3、MD2改进而来，主要增强**算法复杂度**和**不可逆性**；
- MD5算法因其**普遍、稳定、快速**的特点，仍广泛应用于普通数据的加密保护领域。

王卫华



1、MD5算法概述

- 与MD4相比，其设计思想相似，但过程更复杂，提高了**算法的复杂性**，尤其是**非线性函数**的改进，使算法能**抵抗更多的攻击**。

2022/4/24



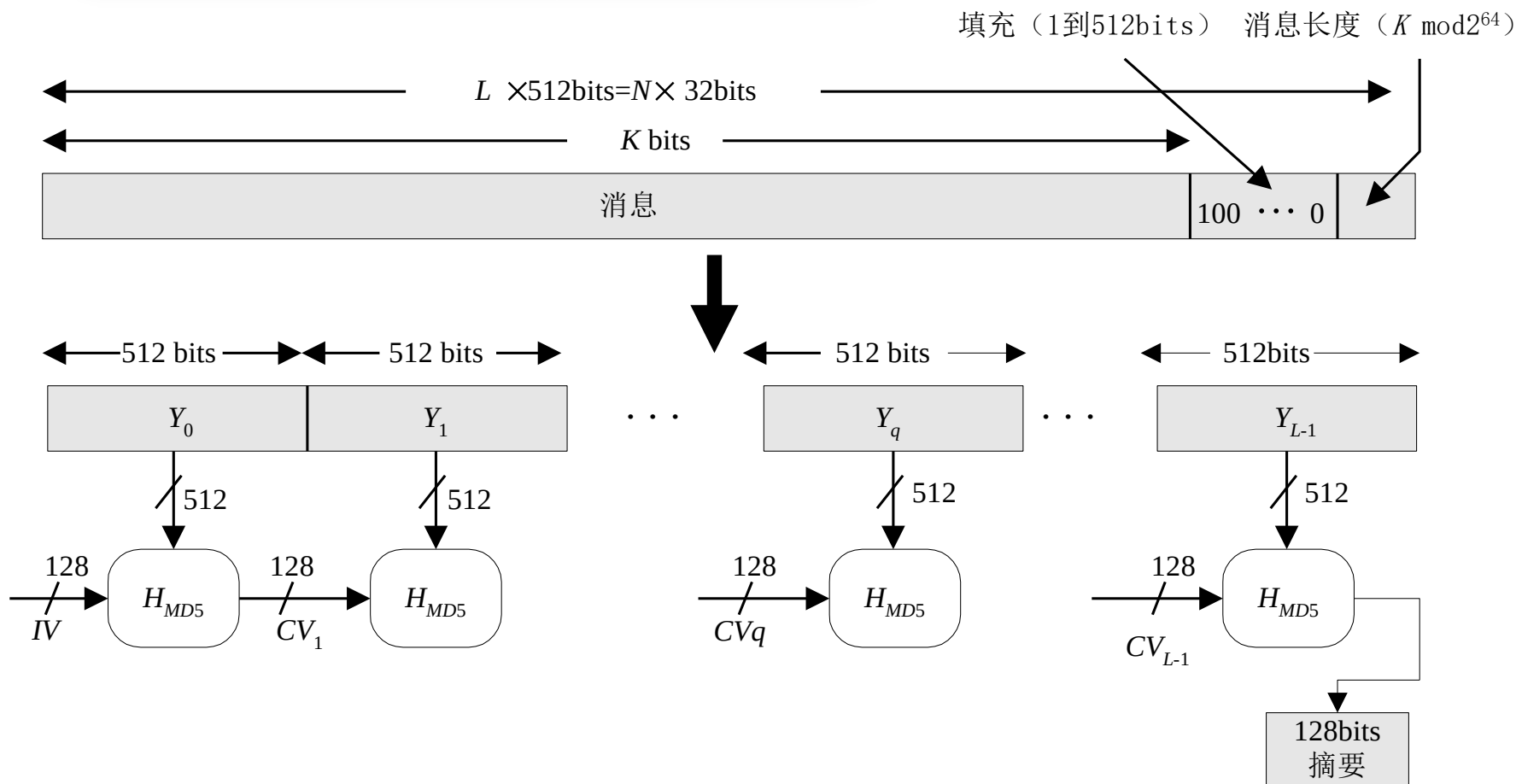
2、MD5算法描述：参数

- MD5算法采用**迭代型哈希函数**的一般结构：
 - 输入为**任意长的消息**
 - 输入分为**512比特的分组**
 - 输出为**128比特的消息摘要**

王卫华



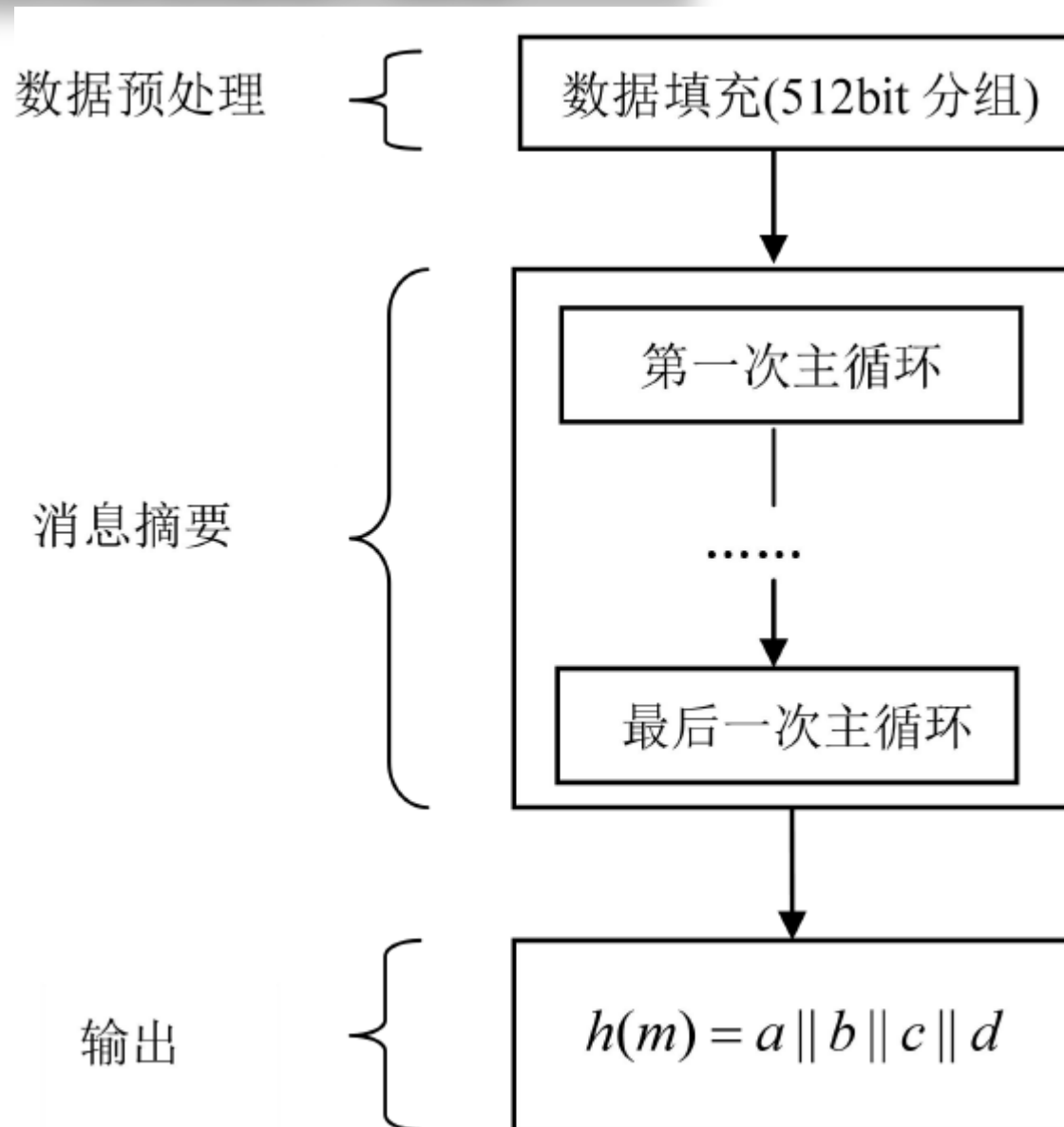
2、MD5算法描述：流程



2022/4/24



2、MD5算法描述：流程





2、MD5算法描述：流程

● 处理过程：1. 对消息填充

① 对消息填充，使得其比特长在模512下为448，即填充后消息的长度为512的某一倍数减64，留出的64比特备用第2步使用。

- 步骤1是必需的，即使消息长度已满足要求，仍需填充。
- 例如，消息长为448比特，则需填充512比特，使其长度变为960，因此填充的比特数大于等于1而小于等于512。
- 填充方式是固定的：第1位为1，其后各位皆为0

王卫华



2、MD5算法描述：流程

Example: 20768 Bit large Message.

- Next multiple of 512: 20992
- $20992 \text{ Bit} - 64 \text{ Bit} = 20928 \text{ Bit}$
- $20928 \text{ Bit} - 20768 \text{ Bit} = 160 \text{ Padding Bits}$
- One "1" Bit and 159 "0" Bits are padded

2024



2、MD5算法描述：流程

- 处理过程：**2. 附加消息的长度**
 - ② 用步骤1留出的64比特以**小端（little-endian）方式**来表示消息被填充前的长度。如果消息长大于 2^{64} ，则以 2^{64} 为模数取模。
 - 小端方式是指按数据的**最低有效字节（byte）（或最低有效位）优先的顺序**存储数据，即将最低有效字节（或最低有效位）存于低地址字节（或位）。
 - 相反的存储方式称为大端（big-endian）方式。

王卫华



2、MD5算法描述：流程

- 处理过程：前2步执行后
 - 消息的长度为512的倍数，设为L倍，则可将消息表示为分组长为512的一系列分组 $Y_0, Y_1, \dots, Y_{L-1}$ 。
 - 而每一分组又可表示为16个32比特长的字，则消息中的总字数为 $N = L \times 16$ ，因此消息又可按字表示为 $M[0, \dots, N-1]$ 。

王卫华



2、MD5算法描述：流程

● 处理过程： 3. 对MD缓冲区初始化

③ 算法使用128比特长的缓冲区以存储中间结果和最终哈希值，缓冲区可表示为四个32比特长的寄存器（A，B，C，D），每个寄存器都以小端方式存储数据：

- 初值取为：A=01234567，B=89ABCDEF，C=FEDCBA98，D=76543210，
- 实际上为：67452301，EFCDAB89，98BADCFE，10325476

王华



2、MD5算法描述：流程

● 处理过程：4. 以分组为单位对消息进行处理

④ 每一分组 $Y_q (q = 0, \dots, L-1)$ 都经一**压缩函数** H_{MD5} 处理。

压缩函数是算法的核心，其中又有**4轮处理过程**

(MD4几轮?) :

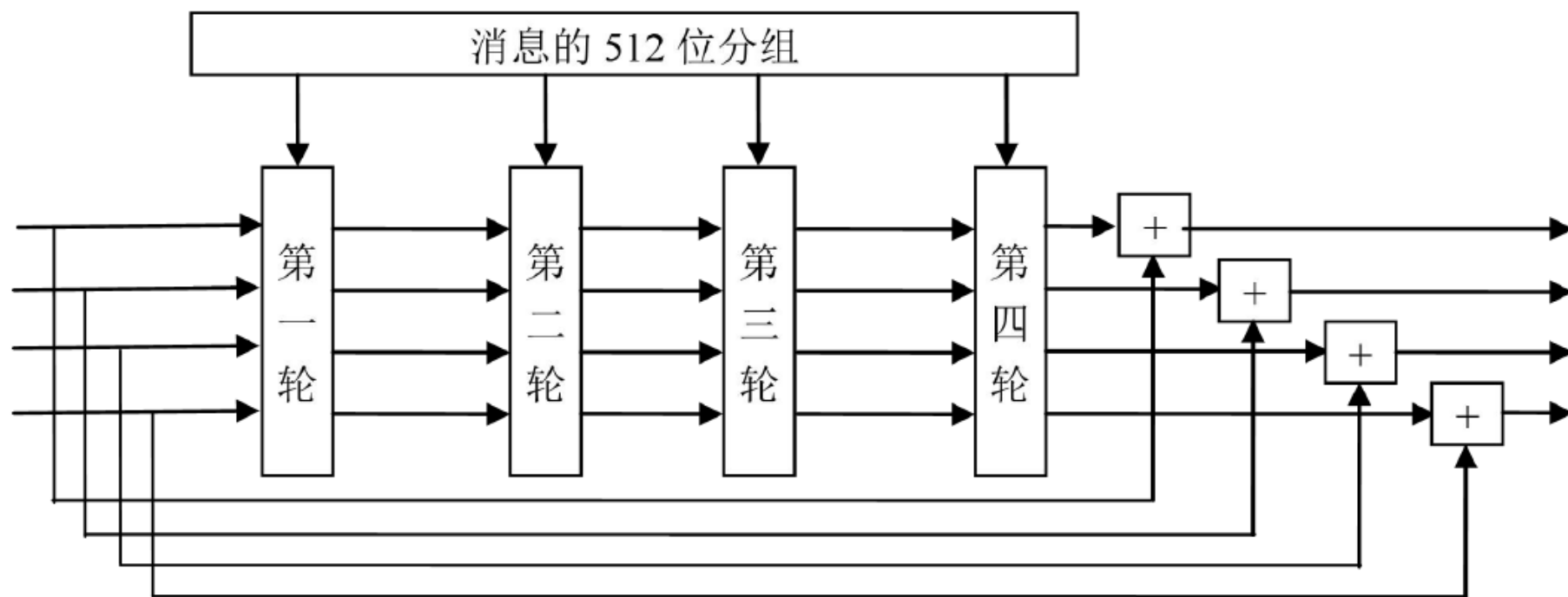
- 每次处理 **512比特的分组**
- 每个分组做**4轮处理**
- 每一轮分为**16步**
- 最后整个消息产生**128比特的输出**，即消息的Hash值

王华



2、MD5算法描述：流程

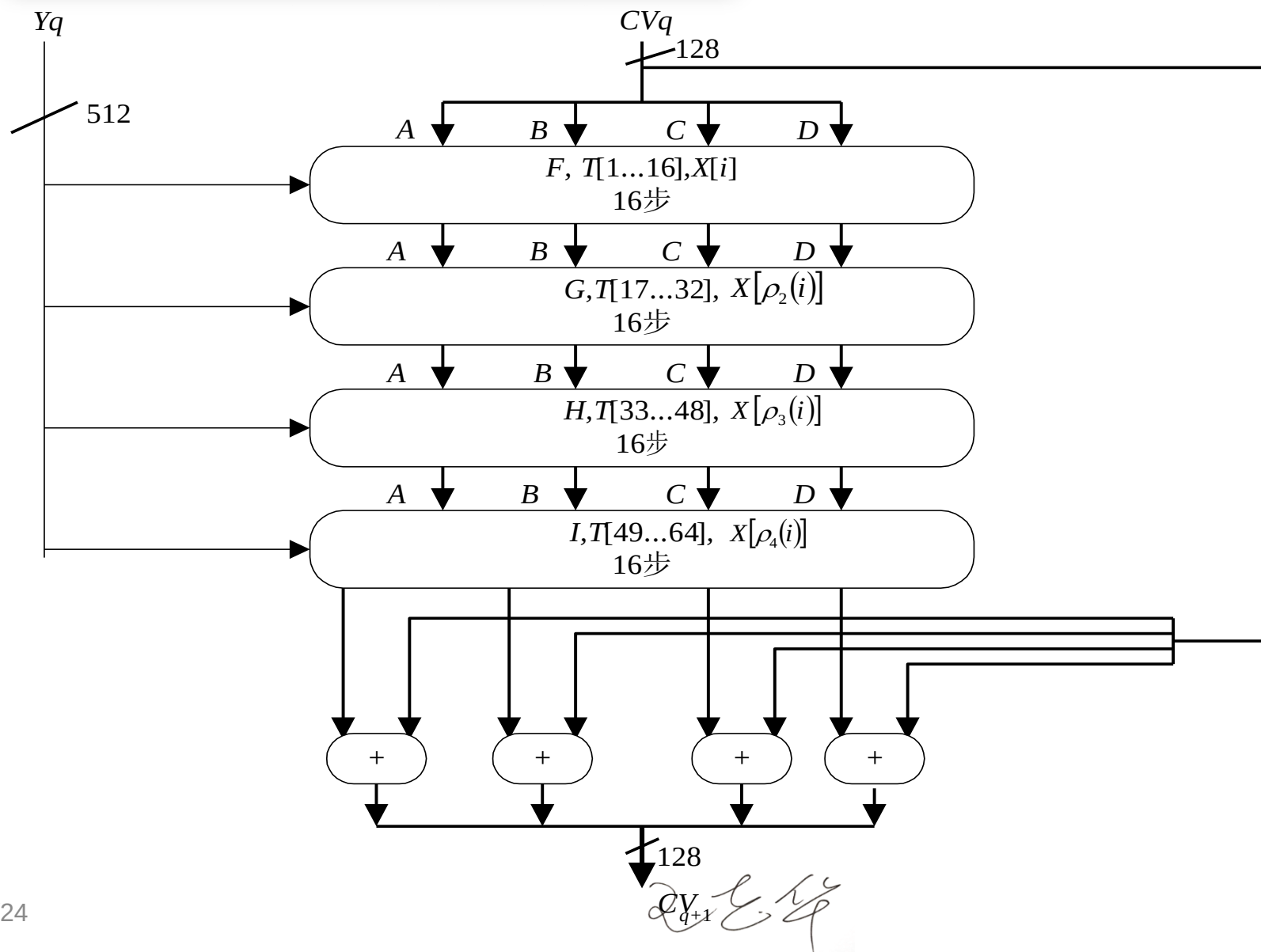
每个分组的处理框图



2022/4/24



2、MD5算法描述：流程





2、MD5算法描述：流程

- 处理过程：4. 以分组为单位对消息进行处理
 - 每轮的输入为当前处理的消息分组 Y_q 和缓冲区的当前值 $A、B、C、D$
 - 每轮的输出仍放在缓冲区中以产生新的 $A、B、C、D$
 - H_{MD5} 的4轮处理过程结构一样，但所用的逻辑函数不同，分别表示为 $F、G、H、I$

2022/4/24



2、MD5算法描述：流程

- 处理过程：**4. 以分组为单位对消息进行处理**
 - 每轮处理过程还需加上**常数表T**中四分之一个元素，分别为 $T[1..16]$, $T[17..32]$, $T[33..48]$, $T[49..64]$
 - 表T有64个元素，**第i个元素T[i]**为 $2^{32} \times \text{abs}(\sin(i))$ 的整数部分，其中sin为正弦函数，i以弧度为单位
 - 由于 $\text{abs}(\sin(i))$ 大于0小于1，所以T[i]可由**32比特的字**表示，即每轮需要16个字，对应16步

王立华



2、MD5算法描述：流程

$T[1]=D76AA478$	$T[17]=F61E2562$	$T[33]=FFFA3942$	$T[49]=F4292244$
$T[2]=E8C7B756$	$T[18]=C040B340$	$T[34]=8771F681$	$T[50]=432AFF97$
$T[3]=242070DB$	$T[19]=265E5A51$	$T[35]=699D6122$	$T[51]=AB9423A7$
$T[4]=C1BDCEEE$	$T[20]=E9B6C7AA$	$T[36]=FDE5380C$	$T[52]=FC93A039$
$T[5]=F57C0FAF$	$T[21]=D62F105D$	$T[37]=A4BEEA44$	$T[53]=655B59C3$
$T[6]=4787C62A$	$T[22]=02441453$	$T[38]=4BDECFA9$	$T[54]=8F0CCC92$
$T[7]=A8304613$	$T[23]=D8A1E681$	$T[39]=F6BB4B60$	$T[55]=FFE47D$
$T[8]=FD469501$	$T[24]=E7D3FBC8$	$T[40]=BEBFBC70$	$T[56]=85845DD1$
$T[9]=698098D8$	$T[25]=21E1CDE6$	$T[41]=289B7EC6$	$T[57]=6FA87E4F$
$T[10]=8B44F7AF$	$T[26]=C33707D6$	$T[42]=EAA127FA$	$T[58]=FE2CE6E0$
$T[11]=FFFF5BB1$	$T[27]=F4D50D87$	$T[43]=D4EF3085$	$T[59]=A3014314$
$T[12]=895CD7BE$	$T[28]=455A14ED$	$T[44]=04881D05$	$T[60]=4E0811A1$
$T[13]=6B901122$	$T[29]=A9E3E905$	$T[45]=D9D4D039$	$T[61]=F7537E82$
$T[14]=FD987193$	$T[30]=FCEFA3F8$	$T[46]=E6DB99E5$	$T[62]=BD3AF235$
$T[15]=A679438E$	$T[31]=676F02D9$	$T[47]=1FA27CF8$	$T[63]=2AD7D2BB$
$T[16]=49B40821$	$T[32]=8D2A4C8A$	$T[48]=C4AC5665$	$T[64]=EB86D391$

表6-3 常数表T

2022



2、MD5算法描述：流程

- 处理过程： 4. 以分组为单位对消息进行处理
 - 第4轮的输出再与第1轮的输入 CV_q 相加，相加时将 CV_q 看作4个32比特的字，每个字与第4轮输出的对应的字按模 2^{32} 相加，相加的结果即为压缩函数 H_{MD5} 的输出

2022/4/24



2、MD5算法描述：流程

- 处理过程： **5.输出**

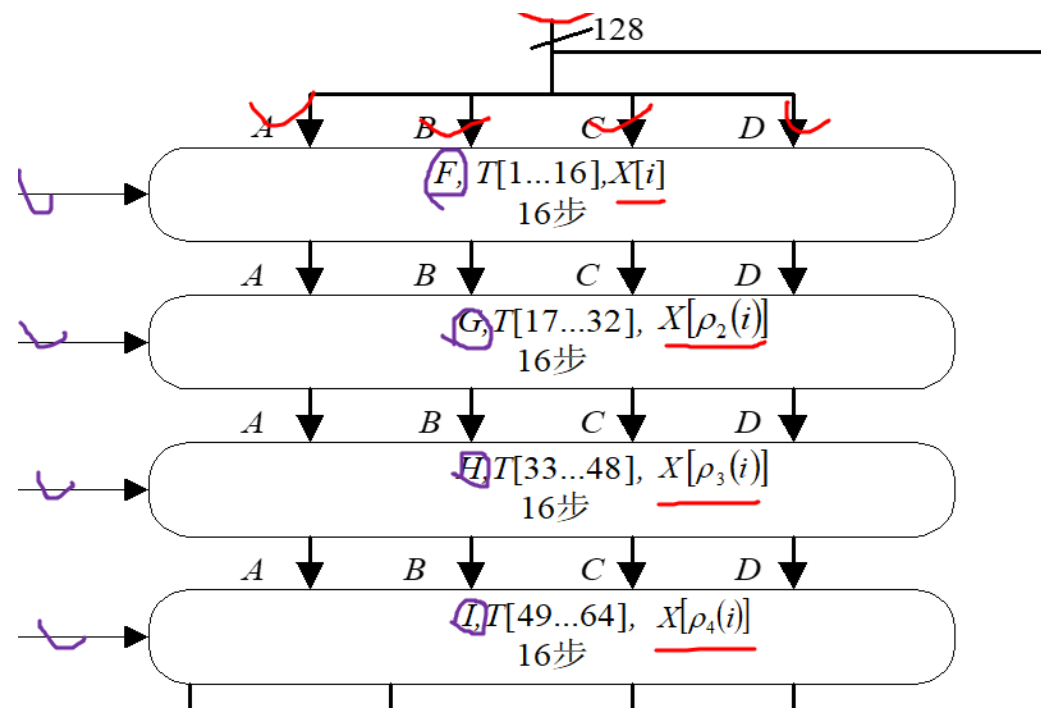
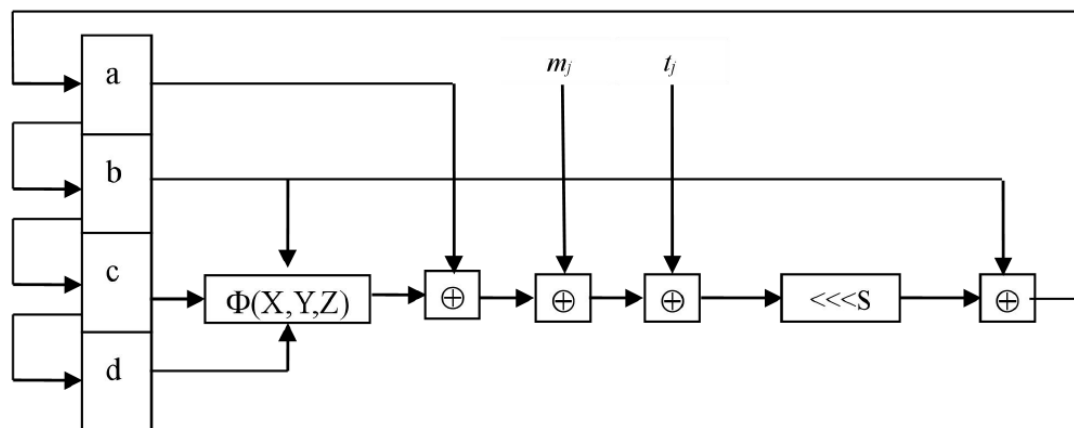
⑤ 消息的L个分组都被处理完后，最后一个压缩函数的输出即为产生的消息摘要。

王华



3、MD5算法的压缩函数

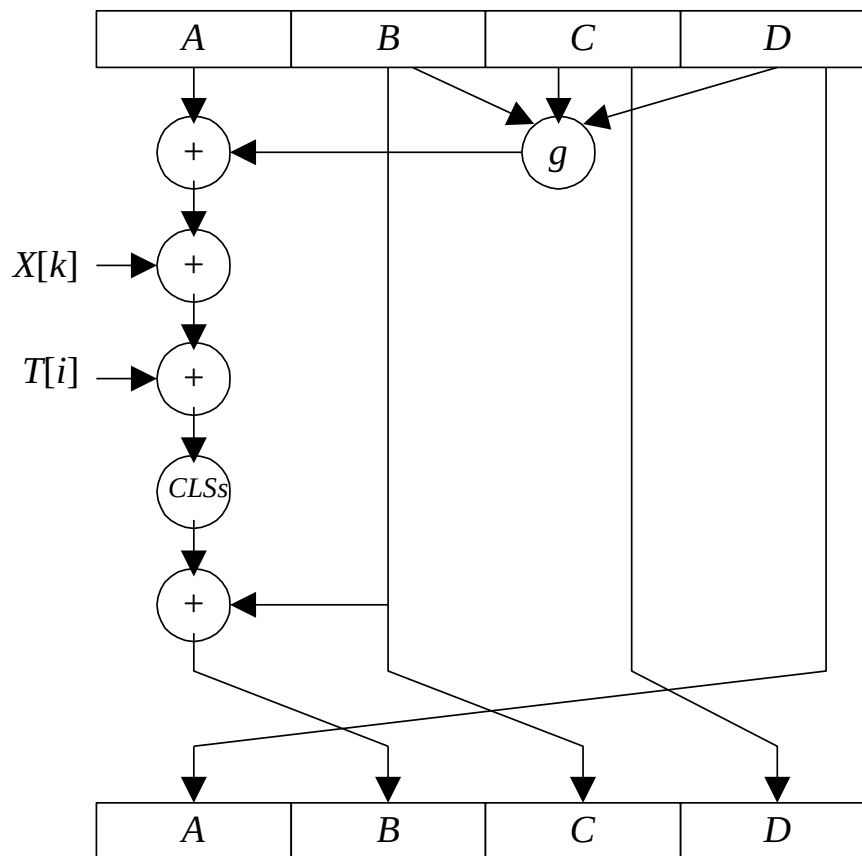
单个操作过程描述



3、MD5算法的压缩函数

- 压缩函数 H_{MD5} 中有**4轮**处理过程，每轮又对缓冲区ABCD进行**16步**迭代运算，**每一步的运算形式**为：

$$a \leftarrow b + CLS_s(a + g(b, c, d) + X[k] + T[i])$$





3、MD5算法的压缩函数

- 压缩函数中的每一步： $a \leftarrow b + CLS_s(a + g(b, c, d) + X[k] + T[i])$
- 其中a、b、c、d为**缓冲区的四个字**，运算完成后再**右循环一个字**，即得这一步迭代的输出。
- g是基本**逻辑函数**F、G、H、I之一。
- CLSs 是**32位存数左循环移s位**，s的取值由表6-4给出。
- T[i]为**表T中的第i个字**，+为**模 2^{32} 加法**。
- $X[k] = M[q \times 16 + k]$ ，即**消息第q个分组中的第k个字**（ $k = 1, \dots, 16$ ）。每个分组有16个字，且 $Y_q (q = 0, \dots, L - 1)$

2022/4/24



3、MD5算法的压缩函数

表6-4 压缩函数每步左循环移位的位数

步数 轮数	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	7	12	17	22	7	12	17	22	7	12	17	22	7	12	17	22
2	5	9	14	20	5	9	14	20	5	9	14	20	5	9	14	20
3	4	11	16	23	4	11	16	23	4	11	16	23	4	11	16	23
4	6	10	15	21	6	10	15	21	6	10	15	21	6	10	15	21

2022/4/24



3、MD5算法的压缩函数

取值如下（不断循环）：

第一轮 7, 12, 17, 22

第二轮 5, 9, 14, 20

第三轮 4, 11, 16, 23

第四轮 6, 10, 15, 21

王华



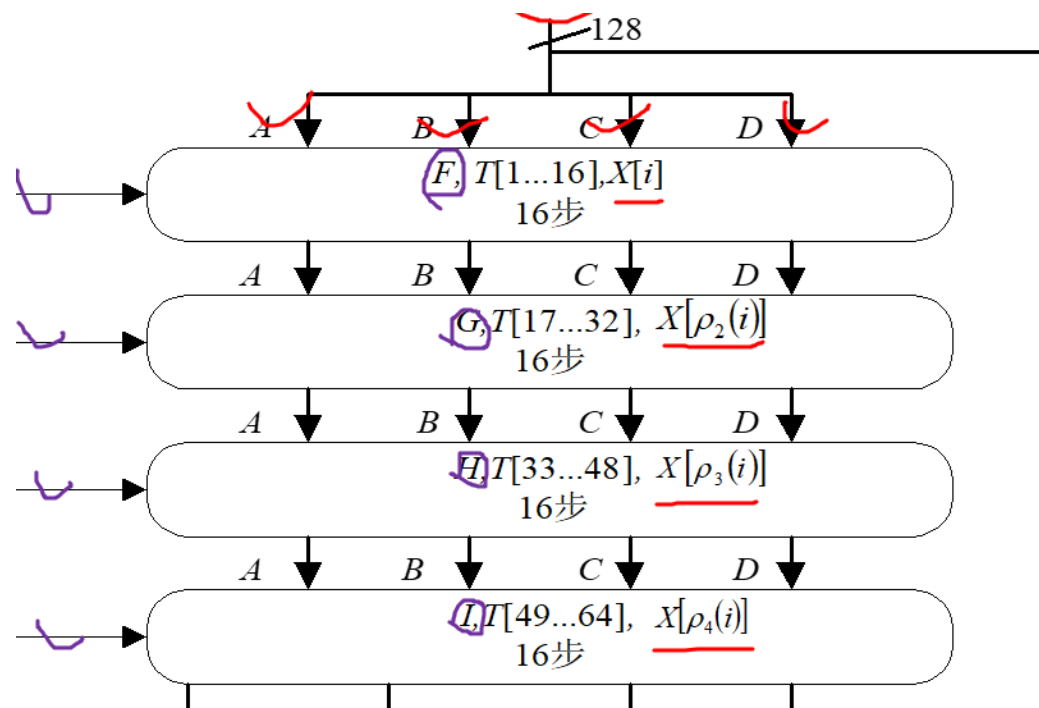
3、MD5算法的压缩函数

- 四轮处理过程中，**每轮以不同的次序使用16个字**：
 - 第一轮以字的初始次序使用。
 - 第二轮到第四轮，分别对字的次序*i*做**置换**后得到一个**新次序**，然后**以新次序使用16个字**。
 - 三个置换分别为：

$$\rho_2(i) = (1 + 5i) \bmod 16$$

$$\rho_3(i) = (5 + 3i) \bmod 16$$

$$\rho_4(i) = 7i \bmod 16$$





3、MD5算法的压缩函数

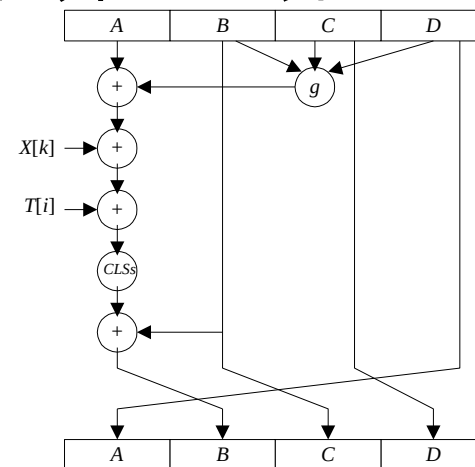
第一轮	自然顺序0-15
第二轮	1,6,11,0,5,10,15,4,9,14,3,8,13,2,7,12
第三轮	5,8,11,14,1,4,7,10,13,0,3,6,9,12,15,2
第四轮	0,7,14,5,12,3,10,1,8,15,6,13,4,11,2,9

王华



3、MD5算法的压缩函数

- 4轮处理过程中，**每轮使用不同的基本逻辑函数g**：
F、G、H、I：
 - 每个逻辑函数的**输入**为3个32比特的字
 - 每个逻辑函数的**输出**是一个32比特的字
 - 每个逻辑函数的**运算**为逐比特的逻辑运算，即输出的第n个比特是**三个输入的第n个比特的函数**
 - 每个逻辑函数的**定义**由表6-5给出，其中 $\wedge$, $\vee$, $\neg$, $\oplus$ 分别是逻辑与、逻辑或、逻辑非和异或运算



2024



3、MD5算法的压缩函数

4轮中的4个非线性函数

表6-5 基本逻辑函数的定义

轮数	基本逻辑函数	$g(b, c, d)$
1	$F(b, c, d)$	$(b \wedge c) \vee (\bar{b} \wedge d)$
2	$G(b, c, d)$	$(b \wedge d) \vee (c \wedge \bar{d})$
3	$H(b, c, d)$	$b \oplus c \oplus d$
4	$I(b, c, d)$	$c \oplus (b \vee \bar{d})$

2022/4/24



3、MD5算法的压缩函数

表6-6 基本逻辑函数的真值表

<i>b</i> <i>c</i> <i>d</i>	<i>F</i> <i>G</i> <i>H</i> <i>I</i>
0 0 0	0 0 0 1
0 0 1	1 0 1 0
0 1 0	0 1 1 0
0 1 1	1 0 0 1
1 0 0	0 0 1 1
1 0 1	0 1 0 1
1 1 0	1 1 0 0
1 1 1	1 1 1 0

王华



3、MD5算法的压缩函数

- 步骤(3)到步骤(5)的处理过程:

$$CV_0 = IV$$

$$CV_{q+1} = CV_q + RF_I[Y_q, RF_H[Y_q, RF_G[Y_q, RF_F[Y_q, CV_q]]]]$$

$$MD = CV_L$$

- 其中IV是步骤(3)所取的缓冲区ABCD的初值,
- Y_q 是消息的第q个512比特长的分组,
- L是消息经过步骤(1)和步骤(2)处理后的分组数,
- CV_q 为处理消息的第q个分组时输入的链接变量 (即前一个压缩函数的输出) ,
- RF_x 为使用基本逻辑函数x的**轮函数**,
- $+$ 为对应字的模 2^{32} 加法,
- MD为最终的哈希值

2022/4/24
王立华



4、MD5的安全性

Rivest猜想作为128比特长的哈希值来说，MD5的强度达到了最大，比如说找出具有相同哈希值的两个消息需执行 $O(2^{64})$ 次运算，而寻找具有给定哈希值的一个消息需要执行 $O(2^{128})$ 次运算。然而，2004年，山东大学王小云等成功找出了MD5的碰撞，发生碰撞的消息是由两个1024比特长的串 M 、 N_i 构成，设消息 $M\|N_i$ 的碰撞是 $M'\|N'_i$ ，在IBM P690上找 M 和 M' 花费时间大约一小时，找出 M 和 M' 后，则只需15秒至5分钟就可找出 N_i 和 N'_i 。

王立华



4、MD5的安全性

- 可找到MD5的碰撞，说明MD5至多是弱碰撞自由的；
- 差分法可用于对单轮的MD5进行分析，但不能对多轮进行分析；
- 王小云已发现MD5有弱点，对任意给定的输入，可以构造出碰撞。
- 目前仍无法找到有指定意义的碰撞，算法仍在使用。

王卫华